MONGODB

SECTION 1

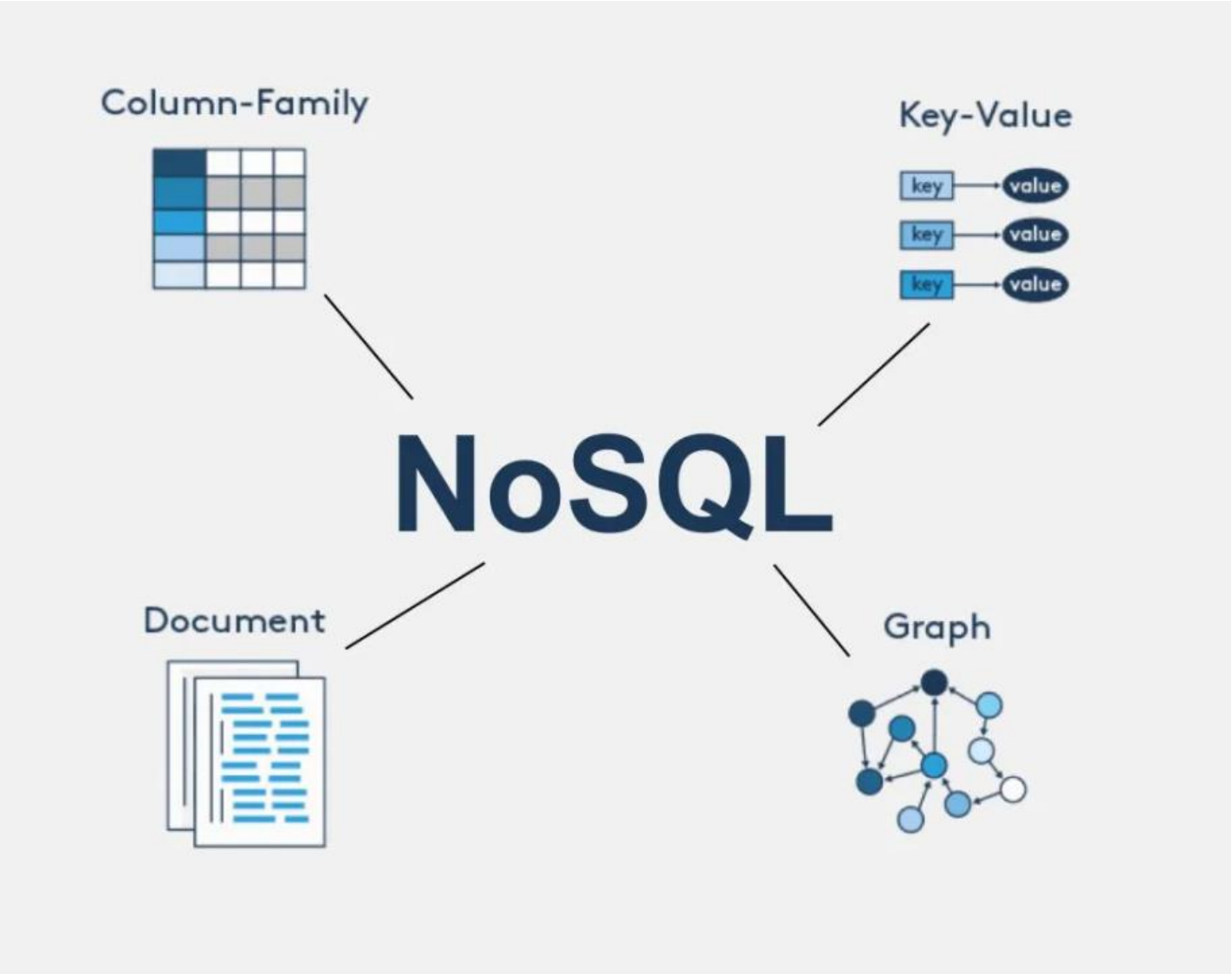
ENG. ASMAA FATHY

NOSQL

- NoSQL is a type of database that differs from traditional databases that rely on tables (which we call relational databases). It is designed to handle massive, distributed data storage, such as data collected by Google or Facebook, which collect terabytes of data every day.

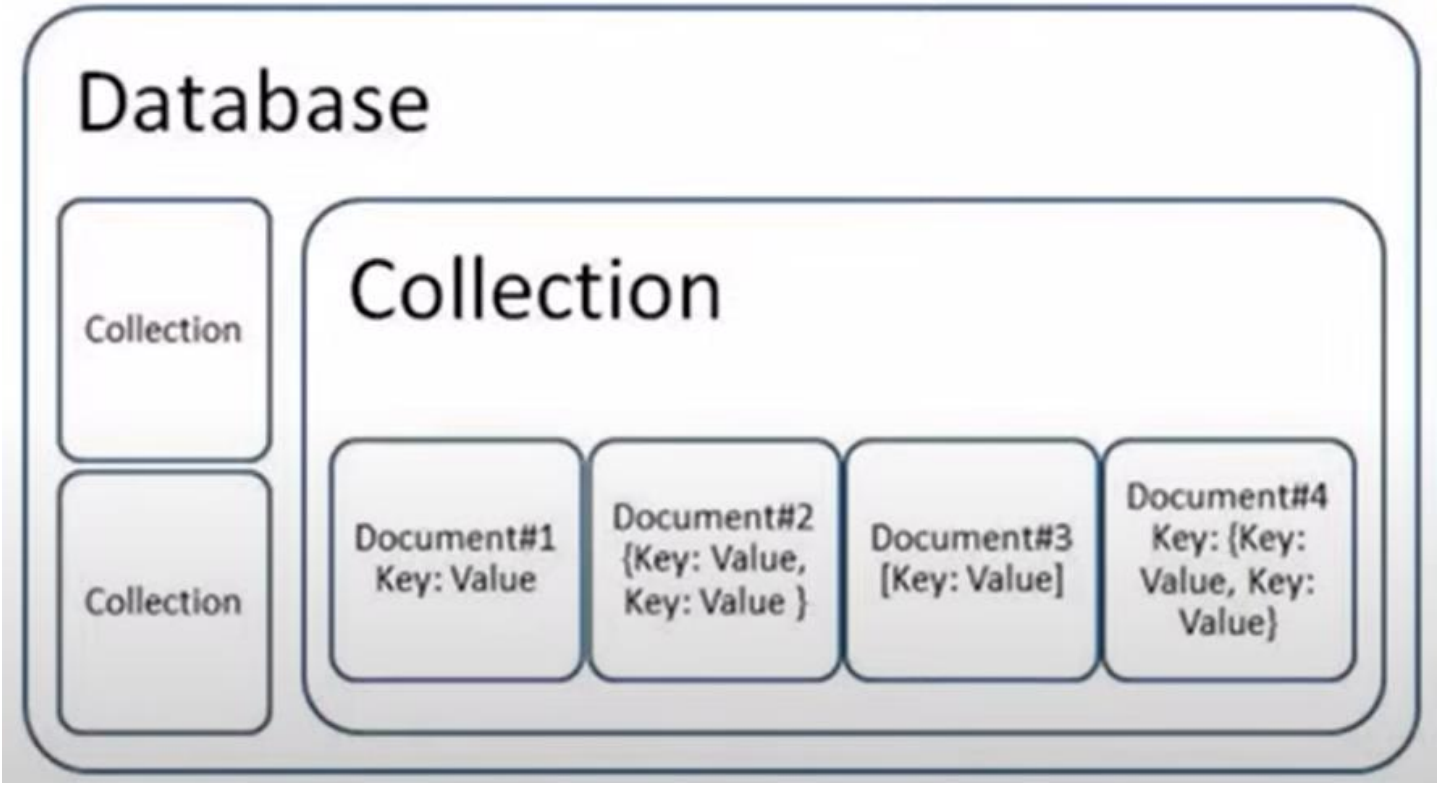
- In NoSQL databases, we do not need to have a fixed data structure (that is, not all data must be in the form of an organized table). We can also avoid complex operations such as joining tables (which we call “JOIN”), which makes it faster and easier to scale as the data grows larger.

NOSQL MODELS



WHAT IS MONGODB ?

- MongoDB is a document-oriented NoSQL database used for high volume data storage. Instead of using tables and rows as in the traditional relational databases.
- MongoDB makes use of collections and documents. Documents consist of key-value pairs which are the basic unit of data in MongoDB. Collections contain sets of documents and function



RDBMS VS MONGODB

RDBMS	MongoDB
Database	Database
Table	Collection
Tuple/Row	Document
column	Field
Table Join	Embedded Documents
Primary Key	Primary Key (Default key _id provided by MongoDB itself)

EXAMPLE: STUDENT TABLE

SID	SFName	SPhone
16s143	Ahmed	95214785
16j7890	Wafa	99663145

Student Collection

Student_16s143 Document

```
{
    SID: 16s143,
    SFName : "Ahmed",
    SPhone : 95214785
}
```

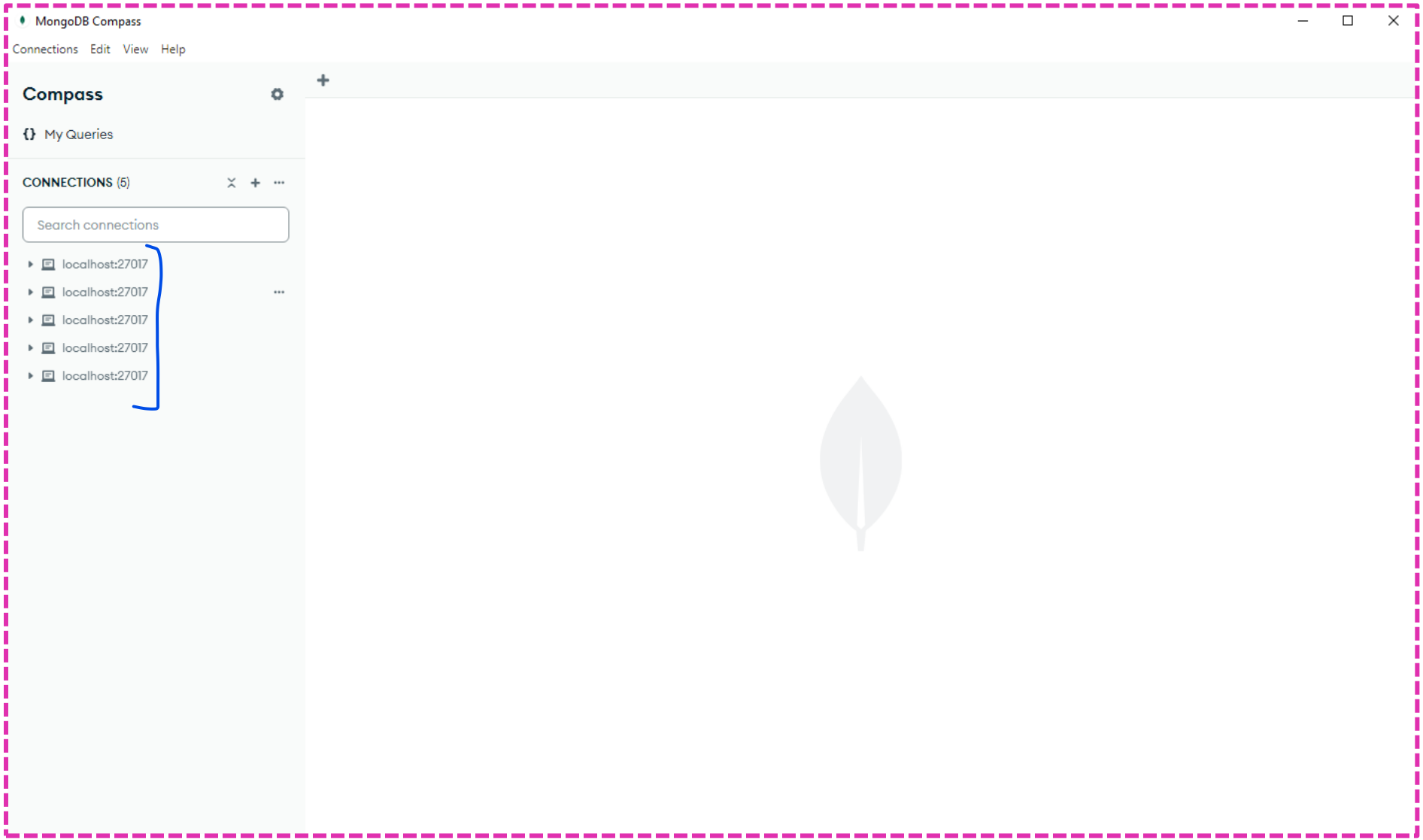
Student_16j7890 Document

```
{
    SID: 16j7890,
    SFName : "Wafa",
    SPhone : 99663145
}
```

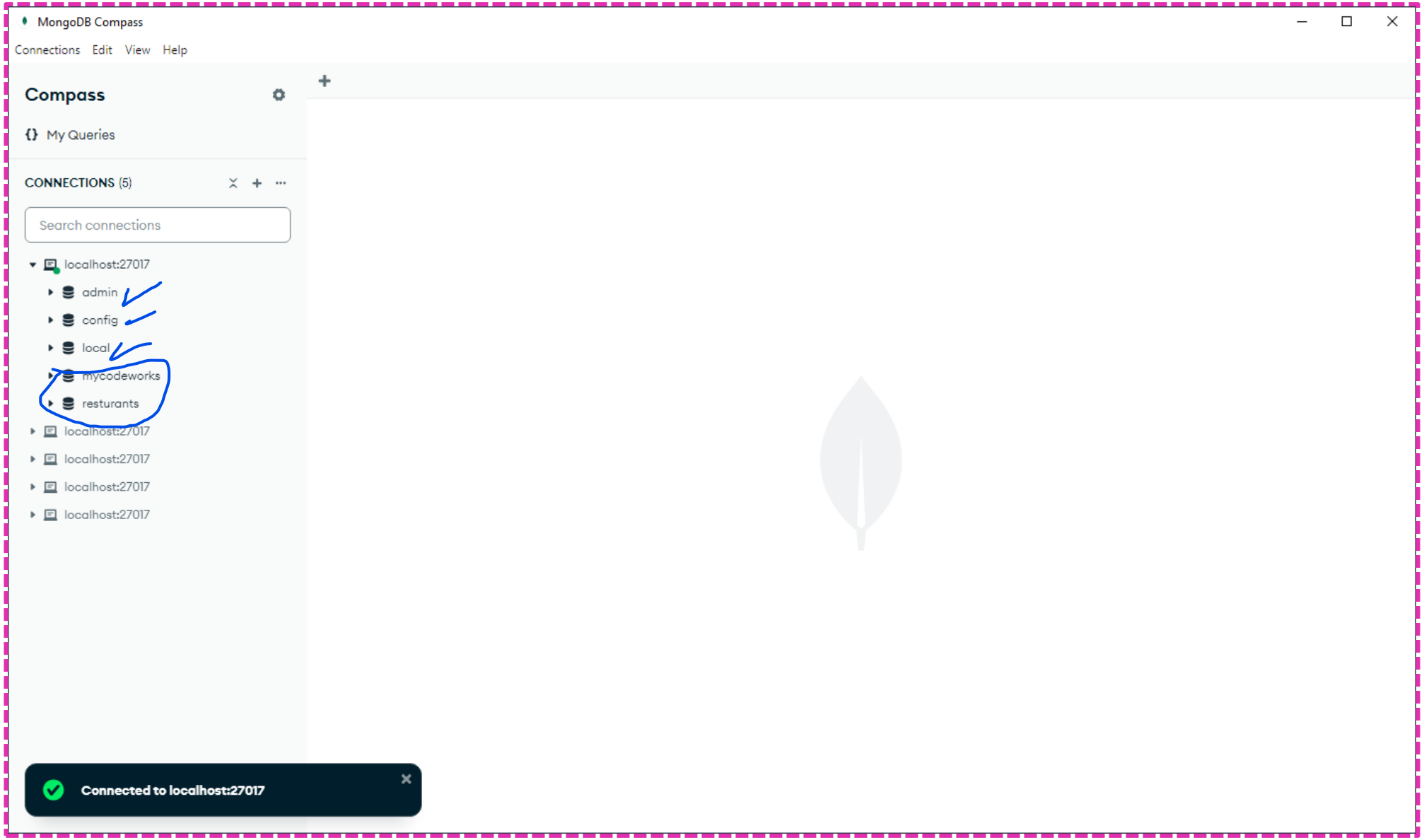
INSTALL MONGODB

- https://youtu.be/NIW2RMWtPbw?si=UE0tdxNs5WwwdvNM
- https://www.mongodb.com/try/download/community
- https://www.mongodb.com/try/download/shell

CONNECT TO MONGODB



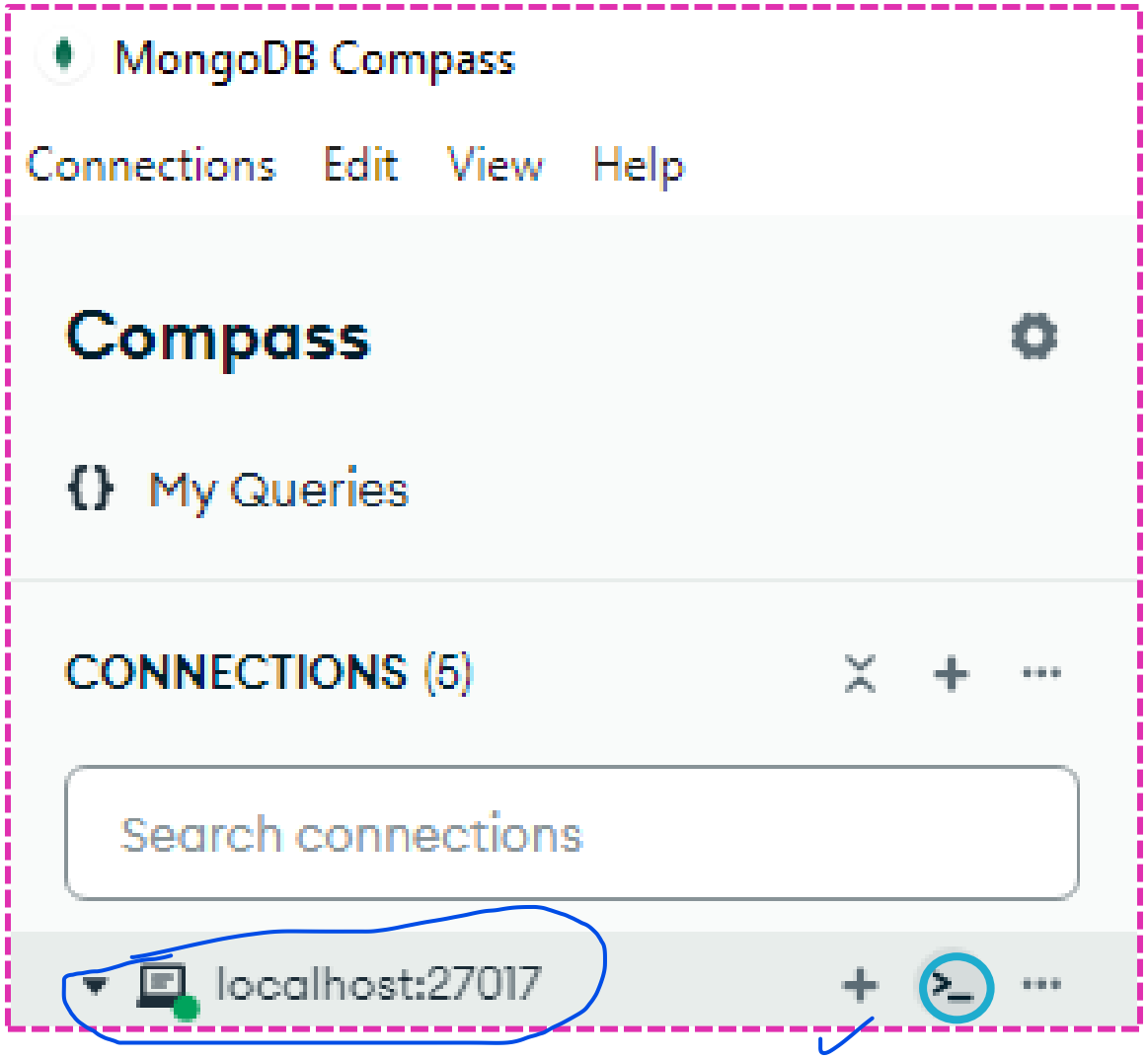
Click on connection for example : localhost:27017


MongoDB Compass
Connections Edit View Help
Compass
My Queries
CONNECTIONS (5)
Search connections
localhost:27017
admin
config
local
mycodeworks
resturants
localhost:27017
localhost:27017
localhost:27017
localhost:27017
Connected to localhost:27017

MONGOSH (MONGODB SHELL)

- MongoDB Shell (mongosh) is a command line interface for interacting with the MongoDB database. It allows users to execute commands and queries on stored data, manage databases and collections, and perform maintenance operations such as backup and restore. Developers use it for quick testing, troubleshooting, and live data management.

TO OPEN MONGOSH



mongosh: localhost:27017

>_MONGOSH

```
test>
```

DISPLAY ALL DATABASES

- To list the databases available to the user, use the helper **show dbs**:

```
>_MONGOSH

> show dbs
< admin          40.00 KiB
  config        108.00 KiB
  local          80.00 KiB
  mycodeworks    40.00 KiB
  resturants    636.00 KiB
test>
```

DISPLAY THE DATABASE YOU ARE USING

- To display the database you are using, type **db**:
- The operation should return test, which is the default database.

```
> db
< test
test>
```

SWITCH DATABASES

- To switch databases, issue the **use <db>** helper, as in the following example:

```
> use resturants
< switched to db resturants
resturants>
```

CREATE A NEW DATABASE AND COLLECTION

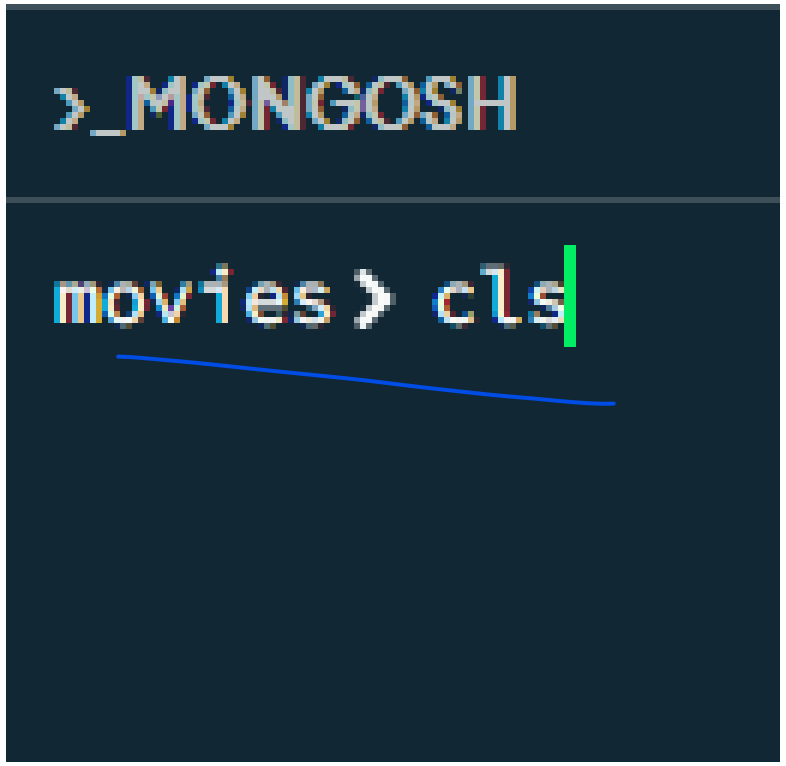
- To create a new database, issue the **use <db>** command with the database that you would like to create. For example, the following commands create both the database movies and the collection comedy using the **insertOne() or insert()** operation:
- If a collection does not exist, MongoDB creates the collection when you first store data for that collection.
- You can create a collection using the **createCollection()** database method.

```
> use movies
< switched to db movies
> db.comedy.insertOne({name:"We are the Millers",year:2013,tagline:"If any one asks", cast:['Jennifer Aniston', 'Luis Guzman', 'Ed Helms', 'Kathryn Hahn'], technical:{runningTime:110, language: 'English'}, m
< {
  acknowledged: true,
  insertedId: ObjectId('6712dd0b1e9e7d0bf0875723')
}
```

```
db.comedy.insert({name: 'Ted', year: 2012, tagline: 'Ted is coming', cast: ['Mark Walhlberg', 'Mila Kunis', 'Seth MacFarlane'],
   technical: {runningTime: 106, language: 'English', prizes: 13, nominations: 27},
   sequel: 'Ted 2,'
   merits: {budget: 50, boxOffice: 535},
   comments:[{by:'Steve', text:'Loved the movie'}, {by:'Dave', text:'Really funny!'}]})
```

CLEAR THE MONGOSH CONSOLE

- The cls command clears the console. You can also clear the console with Ctrl + L and console.clear().



PERFORM CRUD OPERATIONS

CRUD operations:

- Create
- Read
- Update
- Delete

CREATE OPERATIONS

- Create or insert operations add new documents to a collection. If the collection does not exist, create operations also create the collection.
- You can insert a single document or multiple documents in a single operation.
- The MongoDB shell provides the following methods to insert documents into a collection:

a) To insert a single document, use **db.collection.insertOne().**

b) To insert multiple documents, use **db.collection.insertMany().**

INSERT A SINGLE DOCUMENT

- **db.collection.insertOne()** inserts a single document into a collection. If the document does not specify an _id field, MongoDB adds the _id field with an ObjectId value to the new document.
- **insertOne()** returns a document that includes the newly inserted document's _id field value.

```
> use sample_mflix
< switched to db sample_mflix
> db.movies.insertOne(
    {
      title: "The Favourite",
      genres: [ "Drama", "History" ],
      runtime: 121,
      rated: "R",
      year: 2018,
      directors: [ "Yorgos Lanthimos" ],
      cast: [ "Olivia Colman", "Emma Stone", "Rachel Weisz" ],
      type: "movie"
    }
  )
< {
    acknowledged: true,
    insertedId: ObjectId('6712e4231e9e7d0bf0875724')
  }
sample_mflix>
```

```
> db.movies.insertMany([
    {
      title: "Jurassic World: Fallen Kingdom",
      genres: [ "Action", "Sci-Fi" ],
      runtime: 130,
      rated: "PG-13",
      year: 2018,
      directors: [ "J. A. Bayona" ],
      cast: [ "Chris Pratt", "Bryce Dallas Howard", "Rafe Spall" ],
      type: "movie"
    },
    {
      title: "Tag",
      genres: [ "Comedy", "Action" ],
      runtime: 105,
      rated: "R",
      year: 2018,
      directors: [ "Jeff Tomsic" ],
      cast: [ "Annabelle Wallis", "Jeremy Renner", "Jon Hamm" ],
      type: "movie"
    }
])
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6712e5251e9e7d0bf0875725'),
    '1': ObjectId('6712e5251e9e7d0bf0875726')
  }
}
```

INSERT MULTIPLE DOCUMENTS

- **db.collection.insertMany()** can insert multiple documents into a collection. Pass an array of documents to the method. If the documents do not specify an _id field, MongoDB adds the _id field with an ObjectId value to each document.
- **insertMany()** returns a document that includes the newly inserted documents' _id field values.

READ OPERATIONS

- Read operations retrieve documents from a collection; i.e. query a collection for documents.
- You can specify criteria, or filters, that identify the documents to return.
- Use the **db.collection.find()** method in the MongoDB Shell to query documents in a collection.

READ ALL DOCUMENTS IN A COLLECTION

- To read all documents in the collection, pass an empty document as the query filter parameter to the find method. The query filter parameter determines the select criteria
- To return all documents from the sample_mflix.movies collection:
- This operation is equivalent to the following SQL statement:

SELECT * FROM movies

```
>_MONGOSH
> db.movies.find()
< {
    _id: ObjectId('6712e4231e9e7d0bf0875724'),
    title: 'The Favourite',
    genres: [
      'Drama',
      'History'
    ],
    runtime: 121,
    rated: 'R',
    year: 2018,
    directors: [
      'Yorgos Lanthimos'
    ],
    cast: [
      'Olivia Colman',
      'Emma Stone',
      'Rachel Weisz'
    ],
    type: 'movie'
  }
  {
    _id: ObjectId('6712e5251e9e7d0bf0875725'),
    title: 'Jurassic World: Fallen Kingdom',
    genres: [
      'Action',
      'Sci-Fi'
    ],
    runtime: 130,
    rated: 'PG-13',
    year: 2018,
    directors: [
      'J. A. Bayona'
```

SPECIFY EQUALITY CONDITION

- To select documents which match an equality condition, specify the condition as a **<field>:<value>** pair in the query filter document.
- To return all movies where the title equals Tag from the sample_mflix.movies collection:
- This operation corresponds to the following SQL statement:

SELECT * FROM movies WHERE title = “Tag"

```
> db.movies.find( { "title": "Tag" } )
< {
    _id: ObjectId('6712e5251e9e7d0bf08757
    title: 'Tag',
    genres: [
      'Comedy',
      'Action'
    ],
    runtime: 105,
    rated: 'R',
    year: 2018,
    directors: [
      'Jeff Tomsic'
    ],
    cast: [
      'Annabelle Wallis',
      'Jeremy Renner',
      'Jon Hamm'
    ],
    type: 'movie'
  }
```

QUERY OPERATORS

- The following operators can be used in queries to compare values:
  - ✓ $eq: Values are equal
  - ✓ $ne: Values are not equal
  - ✓ $gt: Value is greater than another value
  - ✓ $gte: Value is greater than or equal to another value
  - ✓ $lt: Value is less than another value
  - ✓ $lte: Value is less than or equal to another value
  - ✓ $in: Value is matched within an array

LOGICAL OPERATORS

- The following operators can logically compare multiple queries.
  - ✓ $and: Returns documents where both queries match
  - ✓ $or: Returns documents where either query matches
  - ✓ $nor: Returns documents where both queries fail to match
  - ✓ $not: Returns documents where the query does not match

SPECIFY CONDITIONS USING QUERY OPERATORS

- Use query operators in a query filter document to perform more complex comparisons and evaluations. Query operators in a query filter document have the following form:

  { <field1>: { <operator1>: <value1> }, ... }

- To return all movies from the sample_mflix.movies collection which are either rated PG or PG-13:

```
> db.movies.find( { rated: { $in: [ "PG", "PG-13" ] } } )
< {
    _id: ObjectId('6712e5251e9e7d0bf0875725'),
    title: 'Jurassic World: Fallen Kingdom',
    genres: [
      'Action',
      'Sci-Fi'
    ],
    runtime: 130,
    rated: 'PG-13',
    year: 2018,
    directors: [
      'J. A. Bayona'
    ],
    cast: [
      'Chris Pratt',
      'Bryce Dallas Howard',
      'Rafe Spall'
    ],
    type: 'movie'
  }
```

- This operation corresponds to the following SQL statement:

  SELECT * FROM movies WHERE rated in ("PG", "PG-13")

- **Note :** Although you can express this query using the **$or** operator, use the $in operator rather than the $or operator when performing equality checks on the same field.

SPECIFY LOGICAL OPERATORS (AND / OR)

- A compound query can specify conditions for more than one field in the collection's documents. Implicitly, a logical AND conjunction connects the clauses of a compound query so that the query selects the documents in the collection that match all the conditions.
- To return movies which were released in Mexico and have an IMDB rating of at least 7:

```
> db.movies.find( { countries: "Mexico", "imdb.rating": { $gte: 7 } } )
```

- Use the $or operator to specify a compound query that joins each clause with a logical OR conjunction so that the query selects the documents in the collection that match at least one condition.
- To return movies from the sample_mflix.movies collection which were released in 2010 and either won at least 5 awards or have a genre of Drama:

```
> db.movies.find( {
      year: 2010,
      $or: [ { "awards.wins": { $gte: 5 } }, { genres: "Drama" } ]
  } )
```

LIMIT THE COLLECTION TO JUST TWO

```
> db.movies.find().limit(2)
< {
    _id: ObjectId('6712e5251e9e7d0bf0875725'),
    title: 'Jurassic World: Fallen Kingdom',
    genres: [
      'Action',
      'Sci-Fi'
    ],
    runtime: 100,
    rated: 'PG-13',
    year: 2018,
    directors: [
      'J. A. Bayona'
    ],
    cast: [
      'Chris Pratt',
      'Bryce Dallas Howard',
      'Rafe Spall'
    ],
    type: 'movie',
    movie_id: 5447575
  }
```

THE SKIP() EXPRESSION ALLOWS ONE TO SPECIFY AT WHICH OBJECT THE DATABASE SHOULD BEGIN RETURNING RESULTS.

```
> db.movies.find().skip(1).limit(2)
```

UPDATE OPERATIONS

- The MongoDB shell provides the following methods to update documents in a collection:
- To update a single document, use db.collection.updateOne().
- To update multiple documents, use db.collection.updateMany().
- To replace a document, use db.collection.replaceOne().

UPDATE OPERATOR SYNTAX

- To update a document, MongoDB provides update operators, such as $set, to modify field values.
- To use the update operators, pass to the update methods an update document of the form:

```
{
  <update operator>: { <field1>: <value1>, ... },
  <update operator>: { <field2>: <value2>, ... },
  ...
}
```

- Some update operators, such as $set, create the field if the field does not exist.

UPDATE A SINGLE DOCUMENT

- Use the db.collection.updateOne() method to update the first document that matches a specified filter.
- Example : To update the first document in the sample_mflix.movies collection where title equals “Tag":

The update operation:

- Uses the $set operator to update the value of the plot field for the movie Twilight.
- Uses the $currentDate operator to update the value of the lastUpdated field to the current date. If lastUpdated field does not exist, $currentDate will create the field.

```
>_MONGOSH
> db.movies.updateOne( { title: "Tag" },
  {
    $set: {
      plot: "A teenage girl risks everything-including her life-when she falls in love with a vampire."
    },
    $currentDate: { lastUpdated: true }
  })
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
```

UPDATE MULTIPLE DOCUMENTS

- Use the db.collection.updateMany() to update all documents that match a specified filter.
- Example : To update all documents in the sample_mflix.movies collection to update where runtime is greater than 100:
- The update operation uses the $set operator to update the value of the runtime field to 100.

```
> db.movies.updateMany(
    { runtime: { $gt: 100 } },
    {
      $set: { runtime: 100}
    }
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 3,
    modifiedCount: 3,
    upsertedCount: 0
  }
```

REPLACE A DOCUMENT

- To replace the entire content of a document except for the _id field, pass an entirely new document as the second argument to db.collection.replaceOne()
- When replacing a doument, the replacement document must contain only field/value pairs. Do not include update operators expressions.
- The replacement document can have different fields from the original document. In the replacement document, you can omit the _id field since the _id field is immutable; however, if you do include the _id field, it must have the same value as the current value.
- Example: To replace the first document from the sample_mflix.movies collection where movie_id: 371138:

```
> db.movies.replaceOne(
    { movie_id: 371138 },
    { movie_id: 893421, title: "Tag",
    genres: [
      "Comedy",
    ],
    runtime: 100,
    rated: "A",
    year: 2015,
    directors: [
      "Jeff Tomsic"
    ],
    cast: [
      "Annabelle Wallis",
      "Jeremy Renner",
    ],
    type: "movie", }
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
```

RUN THE FOLLOWING COMMAND TO READ YOUR UPDATED DOCUMENT:

```
> db.movies.findOne( { movie_id: 893421 } )
< {
    _id: ObjectId('6712e5251e9e7d0bf0875726'),
    movie_id: 893421,
    title: 'Tag',
    genres: [
      'Comedy'
    ],
    runtime: 100,
    rated: 'A',
    year: 2015,
    directors: [
      'Jeff Tomsic'
    ],
    cast: [
      'Annabelle Wallis',
      'Jeremy Renner'
    ],
    type: 'movie'
  }
```

DELETE OPERATIONS

- The MongoDB shell provides the following methods to delete documents from a collection:
- To delete multiple documents, use db.collection.deleteMany().
- To delete a single document, use db.collection.deleteOne().

DELETE ALL DOCUMENTS

- To delete all documents from a collection, pass an empty filter document {} to the db.collection.deleteMany() method.
- Example: To delete all documents from the sample_mflix.movies collection:

```
use sample_mflix

db.movies.deleteMany({})
```

- Note: If you want to delete all documents from a large collection, dropping with the db.collection.drop() method. and recreating the collection may have faster performance than deleting documents with the db.collection.deleteMany() method.

DELETE ALL DOCUMENTS THAT MATCH A CONDITION

- You can specify criteria, or filters, that identify the documents to delete. The filters use the same syntax as read operations.
- To specify equality conditions, use <field>:<value> expressions in the query filter document.
- To delete all documents that match a deletion criteria, pass a filter parameter to the deleteMany() method.
- Example: To delete all documents from the sample_mflix.movies collection where the title equals “Tag":

```
> db.movies.deleteMany( { title: "Tag" } )
< {
    acknowledged: true,
    deletedCount: 1
  }
```

DELETE ONLY ONE DOCUMENT THAT MATCHES A CONDITION

- To delete at most a single document that matches a specified filter (even though multiple documents may match the specified filter) use the db.collection.deleteOne() method.
- Example: To delete the first document from the sample_mflix.movies collection where the cast array contains "Olivia Colman":

```
> db.movies.deleteOne( { cast: "Olivia Colman" } )
< {
    acknowledged: true,
    deletedCount: 1
  }
```

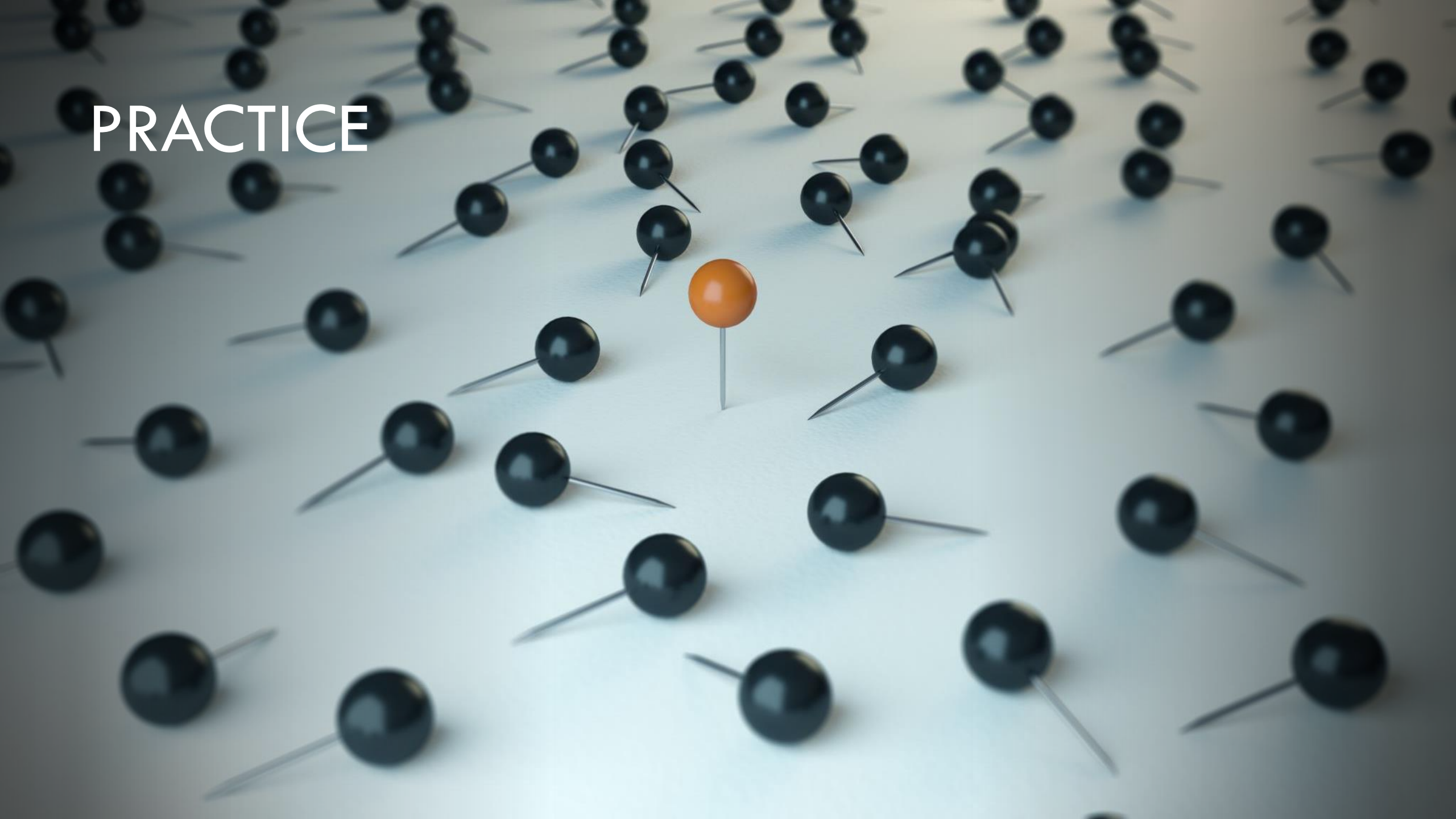
DROP A COLLECTION

- db.collection.drop()

DROP A DATABASE

- use database_name
- db.dropDatabase()

PRACTICE



1. Create a new database called **Companydb**.
2. Create a collection called **Employees**.
3. Enter 3 records in the Employees group containing the following data:

```
{ "name": "Ahmed", "age": 28, "position": "Software Engineer", "salary": 5000 },
{ "name": "Sara", "age": 32, "position": "Project Manager", "salary": 8000 },
{ "name": "Omar", "age": 25, "position": "UI/UX Designer", "salary": 4500 }
```

4. View all employee data within the Employees Group.
5. Show all employees who receive a **salary greater than 4,500**.
6. Show all employees who work as **"Software Engineer" or "UI/UX Designer“.**

SOLUTION

1. use Companydb
2. db.createCollection("Employees")
3. db.Employees.insertMany([
   { "name": "Ahmed", "age": 28, "position": "Software Engineer", "salary": 5000 },
   { "name": "Sara", "age": 32, "position": "Project Manager", "salary": 8000 },
   { "name": "Omar", "age": 25, "position": "UI/UX Designer", "salary": 4500 }])
4. db.Employees.find()
5. db.Employees.find({salary: {$gt: 4500}})
6. db.Employees.find({position: {$in : ["Software Engineer", "UI/UX Designer"]}})
   OR
   db.Employees.find({$or : [{position: "Software Engineer"}, {position: "UI/UX Designer"}]})

THANK YOU